

MVP & Real-Developer Distribution

1. The first objective

Do not start with:

"How do we make this famous?"

Start with:

How do we produce an undeniable before/after result on real developer tasks?

Developer infrastructure becomes interesting when developers can measure that it makes them better.

The MVP must therefore combine:

1. working product;
 2. instrumentation;
 3. real developers;
 4. real repositories;
 5. accepted-task outcomes.
-

2. Ideal first users

Recruit 10–20 power users initially.

Target developers who:

- use Claude Code / Cursor / Codex daily;
- work in non-trivial repositories;
- already delegate multi-file work;
- have experienced repeated agent correction loops;
- can distinguish a truly accepted task from a merely compiling task;
- are comfortable installing local tooling;
- are willing to share anonymized usage metrics.

Good personas:

- technical founders;
- staff/senior engineers;
- indie hackers with active SaaS products;
- AI-native startup engineers;

- open-source maintainers;
- developer-tool engineers.

Avoid initially relying on:

- students doing toy exercises;
 - users who rarely use coding agents;
 - pure greenfield demos;
 - synthetic-only benchmark tasks.
-

3. MVP promise

Do not promise complete autonomy.

Promise something measurable:

Use your existing coding agent. We reduce the amount of supervision needed to get the task accepted.

Possible tagline:

Your coding agent can write the code.

We make sure it understands the job, finishes the job, and proves that it did.

Primary framing:

Intent → Verified Result

4. Initial workflow

The ideal MVP feels almost invisible.

Developer installs:

```
npm install -g [product]  
[product] init
```

or installs a Claude Code plugin.

Then continues working normally:

> Add Google OAuth and reuse our existing onboarding flow.

The product:

1. captures the task;
 2. generates task contract;
 3. retrieves relevant project knowledge;
 4. hands the task to Claude;
 5. observes implementation;
 6. verifies acceptance criteria;
 7. automatically repairs failures;
 8. asks the human only if necessary;
 9. produces an evidence report;
 10. records accepted learning.
-

5. Baseline before improvement

This is essential.

For each beta developer, first observe vanilla agent usage where possible.

Capture:

task
initial prompt
human messages
corrections
agent turns
tool calls
tokens
elapsed time
files changed
tests run
verification outcome
final acceptance

Define:

Human intervention

A human message/action that changes, corrects, clarifies, redirects or verifies the implementation process.

Examples:

- "You forgot the frontend."
- "Do not change that module."
- "Use the existing API."
- "Run the tests."
- "That isn't what I meant."

- manual code fix;
- manual discovery of a failed requirement.

Do not count:

- initial task request;
- optional conversation unrelated to completion;
- final "looks good."

Document this definition carefully so the metric is consistent.

6. Experimental design

For the first 100–500 tasks, compare:

A: Vanilla coding agent

B: Coding agent + our runtime

Metrics:

Metric	Why
human interventions / accepted task	primary autonomy signal
first-pass acceptance	intent quality
total task acceptance rate	output quality
tokens / accepted task	efficiency
time / accepted task	productivity
rework loops	execution quality
verifier catches	direct product value
escaped regressions	safety
developer satisfaction	retention signal

Do not hide negative results.

A strong startup is built by discovering where the system actually works.

7. First cohort recruiting

Founder network

Start with:

- friends;
- former coworkers;
- classmates who are serious developers;
- hackathon contacts;
- startup engineers.

Direct outreach

Message developers who publicly discuss:

- Claude Code workflows;
- Cursor pain;
- AI agent correction loops;
- context management;
- AI code review.

Offer:

"I'm testing a local reliability layer that sits on top of your existing coding agent. I don't need you to change IDEs. I want to measure whether it reduces the number of times you have to correct the agent. Looking for developers who use Claude Code heavily on real repos."

Communities

Useful places:

- Reddit communities for Claude Code, Cursor, programming and AI agents;
- Hacker News;
- X/Twitter developer communities;
- Discords for AI coding tools;
- local developer groups;
- open-source communities.

Do not spam.

Lead with a demo and measured result.

8. The first compelling demo

A demo should show a task where a vanilla coding agent looks successful but misses part of the actual requirement.

Example:

"Add Google OAuth and make new OAuth users use the existing onboarding flow."

Show:

Vanilla

- code implemented;
- tests run;
- agent says done;
- onboarding requirement missed;
- human notices;
- correction;
- agent repairs;
- second issue;
- another correction.

Product

- intent compiler makes onboarding an explicit criterion;
- relevant auth/onboarding context loaded;
- implementation runs;
- verifier detects onboarding failure;
- system automatically repairs;
- human receives one final verified result.

End with metrics:

Vanilla:

human interventions = X

tokens = Y

time = Z

With product:

human interventions = A

tokens = B

time = C

Only use real measured values.

9. Landing page

Hero:

Intent → Verified Result

Sub-head:

Your coding agent can write the code.

We make sure it understands the job, finishes the job, and proves that it did.

Problem section:

Today:

prompt
prompt
correction
prompt
explanation
correction
test
fix
prompt
fix

Future:

intent
↓
verified result

Metric section:

Human interventions per accepted task

Before measurement:

Our goal: drive human interventions per accepted task toward zero.

After measurement:

Display actual cohort data and methodology.

10. Growth loop

Potential organic growth loop:

open-source client/plugin
↓
developer installs
↓
runs on real work
↓
gets personal reliability report
↓
shares improvement
↓
other developers try it
↓
more execution/failure data
↓
product improves

A useful command:

[product] report

Possible output:

Last 30 days

Accepted AI tasks:	84
Human interventions/task:	2.10
First-pass acceptance:	61%
Verification failures caught:	31
Auto-repaired failures:	22
Engineer time saved:	[estimate methodology]
Tokens per accepted task:	[...]

Again, all numbers must be measured.

11. Open-source strategy

Potential strategy:

Open-source:

- local client;
- plugin;
- instrumentation schema;
- basic integrations;
- maybe basic deterministic verifier.

Keep proprietary initially:

- cloud team memory;
- learned verification intelligence;
- task/failure analytics;
- cross-agent performance data;
- advanced context selection;
- organization policy engine.

Benefits:

- developer trust;
- easier installation;
- community integrations;
- transparent handling of code;
- organic distribution.

Do not open-source merely because it is fashionable.

The client should be open if it materially lowers trust friction.

12. Product-led distribution

The best growth asset is a developer-specific before/after report.

Examples:

- "38% fewer corrective turns this week."
- "14 verification failures caught before human review."
- "2.3 fewer human interventions per accepted task."
- "23% fewer tokens per accepted task."

This creates content naturally.

Developers can share:

- screenshots;
 - benchmark runs;
 - before/after traces;
 - task replays.
-

13. Public benchmark

Once the product works on real tasks, create a transparent benchmark.

Possible concept:

Agent Supervision Benchmark

Measure:

- human interventions;
- accepted outcome;
- agent token use;
- elapsed time;
- verification;
- regression rate.

Compare:

Claude alone

Claude + product

Codex alone

Codex + product

Be careful to design it fairly.

The benchmark can become a distribution channel and category-definition tool.

14. User interview questions

Ask behavior questions, not hypothetical sales questions.

Good:

- Walk me through the last task where your coding agent annoyed you.
- How many times did you have to redirect it?
- What did it misunderstand?
- How did you know the task wasn't actually complete?
- What information did you have to repeat?
- What did you manually verify?
- What do you always check before trusting "done"?
- What agent mistakes cost you the most time?
- When do you abandon the agent and fix the code yourself?
- What project knowledge does the agent repeatedly miss?
- Would you install a local tool that observed these workflows?
- What code/data are you unwilling to send to a cloud service?

Bad:

- Would you pay for better AI?
- Do you like this idea?
- Would an intent layer be useful?

15. Beta program structure

Cohort 1 — 5 users

Goal:

Find catastrophic product flaws.

Founder should observe usage closely.

Cohort 2 — 15–20 users

Goal:

Generate enough tasks for comparative metrics.

Cohort 3 — 50–100 users

Goal:

Measure retention and identify repeatable user segment.

At this stage, start testing paid plans.

16. When to charge

Charge once users are relying on it for real work and the product delivers measurable value.

Early options:

- free beta;
- founding-user plan;
- usage credits;
- inexpensive pro tier.

Do not make pricing the experiment before usefulness.

A stronger signal than survey willingness-to-pay:

A developer has used the tool for two weeks, then refuses to work without it.

17. Investment milestone

Do not approach investors with only:

- concept;
- architecture diagram;
- landing page.

Strong pre-seed story:

20 active developers

300 real tasks

baseline vs product comparison

meaningful reduction in interventions

high accepted-task rate

low verifier false-positive rate

strong weekly retention

several paying users or teams asking to pay

Then investor pitch:

Coding models are becoming capable enough that the bottleneck is no longer generation. Engineers are becoming supervisors of AI. We automate that supervisory layer. Across X real tasks, our runtime reduced human interventions per accepted task from A to B while maintaining or improving acceptance and regression rates.

That is fundable evidence.

18. 90-day practical plan

Days 1–14

- define intervention metric;
- build instrumentation;
- instrument own usage;
- gather 20–30 vanilla tasks;
- categorize failure modes.

Days 15–30

- build intent compiler;
- generate task contract;
- test against own tasks;
- recruit first 3–5 external users.

Days 31–45

- add project context retrieval;
- structured project invariants;

- measure incremental effect.

Days 46–60

- deterministic verification;
- acceptance-criteria report;
- begin private alpha.

Days 61–75

- semantic verification;
- repair loop;
- improve privacy/local controls.

Days 76–90

- 10–20 active power users;
- collect 100+ real tasks;
- publish first real before/after results if strong;
- launch landing page;
- open-source/installable client if strategically appropriate;
- begin investor conversations only if evidence is compelling.

19. What "famous" should mean at first

Not mainstream awareness.

The initial target is:

AI-native developers start mentioning the product when someone complains about babysitting Claude/Cursor/Codex.

Category association:

"coding agent keeps saying done when it's not"

↓

"install [Product]"

That is much more valuable than generic traffic.

20. MVP source-of-truth decision

The first MVP exists to prove one claim:

A reliability runtime layered around a strong coding agent can reduce human supervision per accepted engineering task on real repositories.

Everything that does not help prove or falsify that should be deferred.